

Autonomous Agricultural Tractor: Integrated Weed Detection and LiDAR Navigation for Precision Paddy Farming

Benjamin Merryman-Smith, Tony Nguyen, Bilal Dogutas, Krish Shah, Anthony Raphael, Sudip Dhakal

Department of Computing and Software Engineering

Florida Gulf Coast University, Fort Myers, FL 33965, USA

CEN 4930: Introduction to Autonomous Driving Systems, Spring 2026

Abstract—Site-specific weed management in paddy farming offers substantial reductions in herbicide use over conventional broadcast spraying, but field deployment has been limited by three persistent challenges: robust crop-row navigation under canopy where GNSS degrades, real-time visual discrimination between rice and morphologically diverse weeds, and the asymmetric cost of misclassifying rice as weed, which is irreversible.

This paper presents AgriNav, an integrated autonomous tractor system built around four ROS-coupled modules: a custom PyTorch reimplementation of WeedDet for rice detection, a parallel lightweight 1.68M-parameter CNN-FPN variant with asymmetric class weighting, an inverted-logic discrimination module that protects the rice class through a hardcoded confidence-gate veto, and a 6-state constant-velocity-turn-rate Extended Kalman Filter fusing GNSS, IMU, and wheel odometry with three-level outage bridging. Our primary system-level contribution is a four-mechanism LiDAR-camera fusion bridge that uses the navigation LiDAR for region-of-interest constraint, world-coordinate projection, ground-plane filtering, and bidirectional confidence fusion at zero additional hardware cost.

Simulation experiments demonstrate continuous position tracking through a 20-second GNSS outage, crop row detection confidence above 0.9 throughout operation, and rice-detection confidences from 0.32 to 0.95 across paddy, aerial, and post-flood imagery. The LiDAR ROI constraint reduces detection inference region by an estimated 30 to 50 percent.

Index Terms—precision agriculture, autonomous navigation, Extended Kalman Filter, LiDAR, GNSS outage bridging, crop row detection, weed detection, RetinaNet, sensor fusion, ROS

I. INTRODUCTION

Precision agriculture aims to optimize crop yields while minimizing resource waste through targeted interventions. A key challenge in paddy farming is weed management, where conventional broadcast herbicide application wastes chemicals on weed-free areas and risks crop damage. An autonomous tractor capable of navigating crop rows, detecting weeds in real time, and spraying only confirmed weed locations would significantly reduce herbicide usage and labor costs.

Rice is the staple food for more than half of the world's population, and the Food and Agriculture Organization estimates that paddy weed competition can reduce rice yields by 20 to 50 percent if left uncontrolled. The dominant mitigation strategy in production agriculture remains broadcast herbicide application: chemicals are sprayed uniformly across the entire field regardless of where weeds actually grow. This approach is economically inefficient (most of the chemical lands on weed-free

soil), environmentally damaging (runoff carries herbicides into surrounding waterways), and biologically counter-productive (uniform exposure accelerates the evolution of resistant weed populations). Site-specific weed management, in which an autonomous platform identifies individual weed plants and sprays only confirmed targets, has been studied for decades as a path to substantially lower chemical input. Despite that long research history, deployment outside controlled experimental plots remains rare.

Three persistent technical challenges have held back deployment. The first is robust under-canopy localization. As rice matures, the overhead foliage attenuates GNSS signals, introduces multipath reflection from dense vegetation, and in late growth stages can fully occlude the satellite link. A tractor that depends on raw GNSS will lose its global pose estimate at exactly the time of season when targeted weed control is most valuable. The second is real-time visual discrimination between rice and weeds. Weeds in paddy fields are morphologically diverse, share color and texture statistics with the rice crop, and appear at the same scale in the camera frame. A single broadcast classifier struggles to enumerate the species variation while still hitting the latency budget required to spray behind a moving vehicle. The third, and the one that the published literature largely ignores, is the asymmetric cost of misclassification. A false negative on a weed produces a single un-sprayed plant, a recoverable error. A false negative on a rice plant means herbicide gets applied to the crop, which is destructive and irreversible. The error modes are not symmetric in their consequences, but the standard loss formulations and evaluation metrics treat them as if they were.

A. System Overview

This paper presents AgriNav, an autonomous paddy field tractor designed and implemented at Florida Gulf Coast University to address all three challenges in a single integrated system. The architecture is composed of three cooperating modules communicating over the Robot Operating System (ROS). Figure 1 shows the full data flow from raw sensor inputs through perception and localization.

The three modules are: (i) a vision-based object detection subsystem with two parallel implementations (a faithful WeedDet reimplementation and a lightweight CPU-trainable variant)

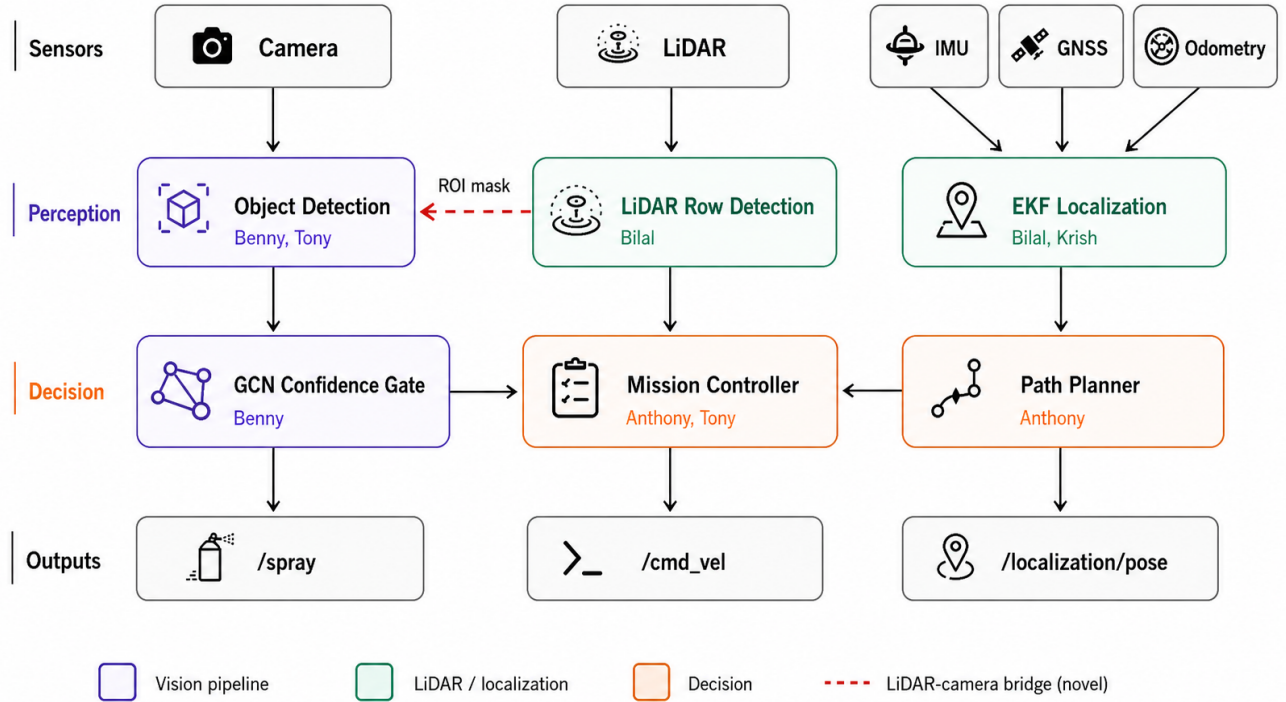


Fig. 1. AgriNav system architecture. Five sensor streams (top) feed three perception and localization modules. The LiDAR row detection module provides both a navigation centerline and a region-of-interest mask back to the object detection module (red dashed arrow), forming the team’s primary system-level novel contribution. Detection results flow through the confidence gate to the spray actuator.

plus an inverted-logic discrimination module that protects the rice class; *(ii)* a LiDAR-based crop row detection module providing both a navigation reference and a region-of-interest mask for the camera pipeline; and *(iii)* a fused localization stack combining a 6-state EKF over GNSS, IMU, and wheel odometry with a separate LiDAR-IMU EKF for under-canopy operation.

The key system-level integration point is the LiDAR-camera bridge: the same 2D LiDAR used for navigation also publishes a projected row-boundary mask to the camera pipeline, which uses it to constrain the inference region. This single channel reduces detection compute by an estimated 30 to 50 percent, removes false positives outside the inter-row zone, and supplies the world-coordinate ground-plane projection needed by the spray actuator. The bridge exploits a sensor that is already on the platform for navigation; the integration cost is software only.

B. Technical Contributions

The technical contributions of this work are:

- **Two parallel weed detection implementations.** A faithful PyTorch reimplementation of the WeedDet architecture (Det-ResNet-50 backbone, efficient FPN, ERetina-Head with Large Separable Convolution, VariFocal + SmoothL1 + GIoU multi-task loss) for accuracy-first GPU training, and a parallel lightweight 1.68M-parameter

CNN-FPN variant with $5\times$ minority-class oversampling and a $3\times$ asymmetric weed-class loss penalty for CPU iteration and embedded deployment.

- **Inverted-logic discrimination with hardcoded rice veto.** A discrimination module that addresses the asymmetric-cost problem by protecting the rice class through a confidence gate that cannot be overridden by any downstream module.
- **LiDAR crop row detection with EKF tracking.** A K-means + RANSAC pipeline for left and right boundary extraction, with each row tracked by an independent two-state EKF that maintains tracking through short LiDAR occlusions in prediction-only mode.
- **6-state CVTR EKF with three-level GNSS outage bridging.** A constant-velocity-turn-rate localization filter fusing GNSS, IMU, and wheel odometry, with explicit AVAILABLE / DEGRADED / OUTAGE status modeling and linearly-inflated process noise during dead reckoning, validated through a 20-second simulated GNSS denial period with bounded positional drift of 2.46 m relative to simulator ground truth.
- **Four-mechanism LiDAR-camera fusion bridge.** The team’s primary system-level novel contribution: region-of-interest masking, world-coordinate projection of bounding boxes, ground-plane filtering, and bidirectional confidence fusion, all using a sensor already present for

navigation at zero additional hardware cost.

- **End-to-end ROS integration.** A defined ROS topic interface contract specifying asymmetric per-class thresholds and timestamped messages for downstream time-aligned fusion.

C. Paper Organization

The remainder of this paper is organized as follows. Section II reviews the foundational work that underpins each module and frames the differences between published approaches and our system design. Section III presents the WeedDet object detection implementation. Section IV presents the lightweight CNN-FPN detection variant and the ROS 2 detection-to-discrimination interface. Section V covers LiDAR-only crop row navigation. Section VI discusses EKF-based GNSS-IMU sensor fusion. Section VII details the LiDAR-IMU EKF row following system for under-canopy navigation. Section VIII describes the full system integration including the LiDAR-camera bridge. Section IX provides an honest accounting of current limitations and known faulty components. Section X discusses real-world deployment scenarios and the work required to reach them. Section XI concludes.

II. RELATED WORK

The AgriNav system draws on foundational publications that together define the technical landscape it builds in. This section summarizes what each work proposed, how it solved its target problem, and how the AgriNav design adopts, modifies, or departs from that approach.

A. WeedDet: Improved RetinaNet for Paddy Weed Detection

Peng et al. [1] addressed the problem of real-time, multi-class weed detection in paddy field imagery. Their starting point was that off-the-shelf single-stage detectors such as RetinaNet [2] and YOLOv3 underperformed on small, densely-packed plant instances, and that two-stage detectors achieved competitive accuracy only at frame rates incompatible with field-rate deployment behind a moving tractor. They proposed three structural modifications to baseline RetinaNet. First, they replaced the standard ResNet-50 [3] stem (which applies a 7×7 stride-2 convolution and a max-pool that together discard fine spatial detail) with a custom Det-ResNet-50 stem that uses two stacked 3×3 convolutions and a residual block, preserving high-resolution spatial information for small-object detection. Second, they reduced the standard five-level FPN [4] to three pyramid levels (eFPN), dropping P6 and P7 since multi-octave extreme-scale handling is unnecessary in paddy imagery; this saved 7.71 million parameters. Third, they replaced the four 256-channel head convolutions with a single 64-channel reduction followed by a Large Separable Convolution that factorizes a $k \times k$ kernel into $k \times 1$ and $1 \times k$ pairs. They also replaced standard Focal Loss with VariFocal Loss [7], which resolves a known training-inference inconsistency by training with IoU as the positive-class target. The reported result was 94.1 percent mAP on a custom 9-class paddy field dataset at 24.3 frames per second.

Our approach. We faithfully reimplement the WeedDet architecture in PyTorch (Section III), but adapt it to a single-class rice detection task as part of the inverted-logic discrimination strategy described below. We additionally develop a parallel lightweight 1.68M-parameter CNN-FPN variant (Section IV) that retains the multi-scale FPN aggregation but uses a custom 5-stage backbone for CPU-trainable iteration and embedded deployment. Neither Peng et al. nor any other detection paper in the agricultural literature addresses the asymmetric cost of false negatives on the crop class; we add a hardcoded confidence-gate veto that cannot be overridden by any downstream module.

B. LiDAR-Based Crop Row Detection

Liu et al. [8] addressed over-canopy autonomous navigation in agricultural fields, where conventional vision-based row following becomes unreliable under variable lighting, shadow, and vegetation density. They proposed a pipeline that first segments LiDAR point clouds into left and right crop row clusters using K-means clustering, then fits a line to each cluster using RANSAC to extract the row geometry, and finally tracks each row over time using an Extended Kalman Filter. The system was validated on a full-scale Amiga robot in Gazebo simulated fields with corn and soybean crops at multiple growth stages, including challenging scenarios with weeds and row discontinuities.

Our approach. We adopt the K-means + RANSAC + EKF skeleton directly (Section V and Section VII), but apply it to paddy field environments under the canopy where GNSS degrades. Each row is tracked by an independent two-state EKF with state $\mathbf{x} = [d, \phi]^T$, where d is the perpendicular distance from the robot to the row and ϕ is the row angle relative to the robot heading. Beyond the navigation use of the centerline, we also project the detected row boundaries into the camera's image plane and publish them as a region-of-interest mask consumed by the camera-based detector, which is not addressed in the original Liu et al. work.

C. Multi-Sensor Fusion for Under-Canopy Row Following

Higuti et al. [9] addressed reliability of autonomous row following for compact agricultural robots operating under canopy. They demonstrated that fusing IMU data with LiDAR measurements through an Extended Kalman Filter substantially improves navigation continuity under sensor noise, partial occlusion, and short row gaps. The headline empirical result is that adding EKF-based sensor fusion improved the mean distance between manual interventions from 51.6 m without EKF to 400 m with EKF in fields without major gaps, an approximately $7.7\times$ improvement.

Our approach. We adopt the same fusion philosophy but extend it with GNSS as a third source (Section VI) and add explicit GNSS outage bridging logic (Section VII). The 6-state constant-velocity-turn-rate model fuses three asynchronous streams (50 Hz IMU, 20 Hz odometry, 1 Hz GNSS when available) and inflates process noise linearly during dead reckoning to reflect the growing uncertainty from gyroscope

drift and wheel slip. When GNSS returns, the Kalman gain naturally re-incorporates the absolute position measurements without requiring a manual reset.

D. LIO-EKF: High-Frequency LiDAR-Inertial Odometry

Wu et al. [10] addressed the question of whether classical EKF-based methods remain competitive with modern factor-graph and optimization-based approaches for LiDAR-inertial odometry. They proposed LIO-EKF, a lightweight system based on point-to-point registration with a classical EKF scheme that achieves pose estimation at close to the IMU frame rate (100 Hz) while maintaining accuracy comparable to more complex methods. The result confirms that classical EKF formulations remain a strong choice for real-time robotic applications when computational simplicity and predictable latency are required.

Our approach. We do not implement LIO-EKF directly, but the result motivates our overall architectural choice to use classical EKFs throughout the localization stack rather than a sliding-window optimizer or factor graph. Section VII describes how we apply this principle to the GNSS-IMU-odometry fusion problem, achieving comparable robustness for the under-canopy paddy environment with simpler tuning and predictable real-time performance.

E. Foundational Detection Components

Several other works are used directly as building blocks rather than being modified. Smooth L1 loss [5] provides outlier-robust bounding box regression. Generalized IoU loss [6] provides non-zero gradient even when predicted and ground-truth boxes do not overlap, complementing Smooth L1 in early training. The DeepWeeds dataset and benchmarking work of Olsen et al. [23] establishes the typical performance envelope for comparable architectures and motivates our expected performance bounds at higher training resolution.

III. WEEDDET: IMPROVED RETINANET FOR REAL-TIME PADDY WEED DETECTION

The detection logic developed for AgriNav is intentionally inverted from typical weed detection pipelines. Rather than enumerating weed species, the system identifies the single rice class with high confidence and treats anything outside high-confidence rice regions as a weed. The reasoning is asymmetric risk: a false negative on a rice plant results in herbicide damage to the crop, which is categorically worse than a missed weed. Neither of the foundational papers in this area addresses this asymmetry in their loss formulation, evaluation metrics, or actuation policy.

A. WeedDet Architecture

The detection model is a custom PyTorch implementation of WeedDet, an improved single-stage object detector based on RetinaNet, proposed by Peng et al. [1] and adapted here to a single-class rice detection task. The architecture introduces three structural modifications to baseline RetinaNet.

Det-ResNet-50 backbone. The standard ResNet-50 [3] stem applies a 7×7 convolution with stride 2 followed by a

max-pool, which together reduce input resolution by a factor of four in the first stage. While acceptable for whole-image classification, this aggressive early downsampling discards fine-grained spatial information needed for detecting small, densely-packed plant instances. Det-ResNet-50 replaces the 7×7 stride-2 convolution and max-pool with two stacked 3×3 convolutions and a residual block. Channel widths in the stem are reduced from 64 to 16 and then 32, holding the additional compute cost negligible. The reported ablation in [1] shows mAP improving from 88.6% to 91.4% at essentially the same FLOPs.

Efficient Feature Pyramid Network (eFPN). The neck is a three-level feature pyramid network [4]. Lateral 1×1 convolutions reduce C3, C4, and C5 each to 256 channels. Top-down upsampling and additive fusion produce P3, P4, and P5. The standard RetinaNet P6 and P7 levels are intentionally omitted. Their omission saves approximately 7.71 million parameters without measurable accuracy loss on plant-scale objects, where multi-octave extreme-scale handling is unnecessary.

Enhanced RetinaNet Head (ERetinaHead). The detection head reduces the standard RetinaNet four 3×3 256-channel convolutions to a single 1×1 convolution that drops channels from 256 to 64, followed by a Large Separable Convolution that factorizes a $k \times k$ kernel into a $k \times 1$ then $1 \times k$ pair. With $k = 7$, the receptive field is 7 pixels in each spatial direction at $2 \cdot k \cdot c$ parameters rather than $k^2 \cdot c$, a $3.5 \times$ reduction in spatial-mixing parameters.

Anchor configuration. Nine anchors are defined per spatial location across each pyramid level. The anchors span three aspect ratios, $\{1:2, 1:1, 2:1\}$, and three octave scales, $\{2^0, 2^{1/3}, 2^{2/3}\}$. The base anchor scale is set to 6, following the value selected by ablation in the original WeedDet paper.

B. Loss Formulation

The total loss is a sum of three components: SmoothL1 and GIoU on the regression branch, and VariFocal on the classification branch.

Smooth L1 loss [5] is applied to the encoded box deltas. For a single coordinate x ,

$$\text{SmoothL1}(x) = \begin{cases} 0.5x^2 & \text{if } |x| < 1 \\ |x| - 0.5 & \text{otherwise.} \end{cases} \quad (1)$$

The Generalized IoU loss [6] provides non-zero gradient even when predicted and ground-truth boxes do not overlap. Let $\tilde{x}$ denote the predicted box, x the ground truth, and u the smallest enclosing box. The GIoU is

$$\text{GIoU}(\tilde{x}, x) = \frac{|\tilde{x} \cap x|}{|\tilde{x} \cup x|} - \frac{|u \setminus (\tilde{x} \cup x)|}{|u|}. \quad (2)$$

VariFocal Loss [7] replaces standard Focal Loss [2] in the classification head. For positive samples, the classification target is the IoU between predicted and ground-truth box rather than a binary 1. With predicted probability p and target q ,

$$\text{VFL}(p, q) = \begin{cases} -q[q \log p + (1 - q) \log(1 - p)] & \text{if } q > 0 \\ -\alpha p^\gamma \log(1 - p) & \text{if } q = 0, \end{cases} \quad (3)$$

with $\alpha = 0.75$ and $\gamma = 2.0$.

For each ground-truth box, IoU is computed against all anchors. An anchor is labeled positive if its maximum IoU exceeds 0.5, negative below 0.4, and ignored otherwise. The 0.4-to-0.5 ignore band reduces training instability with small batch sizes.

C. Data Pipeline

A single rice detection dataset was sourced from Roboflow [19], containing approximately 4,041 images with tightly annotated per-plant bounding boxes. A secondary dataset of 1,347 augmented images was generated through horizontal flips and noise injection. Both datasets are exported in Pascal VOC XML format.

The pipeline is hybrid. Dataset consolidation uses pre-annotated Roboflow exports, while validation, bounding-box filtering, and sample rejection occur dynamically during training. Stage 1 extracts archives and consolidates images and XML files into flat `images/` and `annotations/` directories with basename-matching. Stage 2 parses XML annotations, casts bounding box coordinates to floating point, and removes degenerate boxes (width or height ≤ 1 pixel). Images with no valid bounding boxes are excluded to prevent learning from negative-only samples. Stage 3 prepares paired image-annotation samples, resizes images to 600×1000 , and scales bounding boxes accordingly.

D. Training Configuration and Results

A baseline torchvision `retinanet_resnet50_fpn` was trained first as a sanity check on the data pipeline. Hyperparameters: SGD with learning rate 0.001, momentum 0.9, weight decay 10^{-4} , batch size 2, StepLR with multiplicative factor 0.1 after epoch 9, gradient clipping at max norm 1.0, and 12 total epochs. The baseline converged to an average loss of approximately 0.63.

The custom WeedDet model was trained for 94 epochs across multiple Google Colab sessions due to runtime limits. Configuration: SGD, initial learning rate 0.01, momentum 0.9, weight decay 10^{-4} , batch size 2, gradient clipping at max norm 1.0. A cosine learning rate schedule with warm restarts was applied. Training data was augmented with horizontal flip, random shift, random brightness, Gaussian blur, and CLAHE. Training stopped when validation loss diverged from training loss, indicating overfitting onset. Best loss observed was approximately 1.2346.

Quantitative mAP evaluation was not completed within scope; this is acknowledged as a limitation. Validation was qualitative. On a dense ground-level paddy field image with overlapping rice stalks, the trained model produced approximately 60 detected rice instances at confidence scores 0.32 to 0.88. On a sparse aerial-perspective image with well-separated young rice on light gravel, approximately 22 detections at 0.33 to 0.77. On a post-flood image with high water-mud contrast, detections at 0.40 to 0.95.

E. Discrimination Module: Rice Protection by Inversion

The discrimination module sits between the detection model and the spray actuator. It converts a list of detections into a herbicide-actuation decision per nozzle. Any image region not covered by a high-confidence rice bounding box is classified as a candidate weed zone.

Before any spray valve opens, every actuation candidate must pass through a confidence gate that imposes two conditions in conjunction. First, the rice detection's IoU-Aware Classification Score must exceed 0.5. Second, the predicted class must not equal rice. This second condition is the hard-coded rice veto, implemented as an unconditional return of `spray = False` whenever the rice class is predicted at any confidence level. The veto cannot be overridden by any downstream module.

An optional Excess Green minus Excess Red (ExGR) preprocessing transform is available:

$$\text{ExGR} = 3G - 2.4R - B, \quad (4)$$

with normalized channel values in $[0, 1]$. The transform boosts contrast of green plant material against soil and water backgrounds. ExGR is treated as a tunable preprocessing option rather than a fixed component, since its benefit depends on ambient lighting.

IV. LIGHTWEIGHT CNN-FPN DETECTION PIPELINE AND ROS 2 INTERFACE

A. Motivation

The full WeedDet reimplementation described in Section III faithfully follows the architecture of Peng et al. [1] and is designed for accuracy-first GPU training. For deployment validation, dataset experimentation, and the ROS 2 integration work that connects detection to the downstream discrimination module, a parallel lightweight implementation was developed. This implementation has three goals: (1) a parameter-count small enough to train on commodity CPU hardware while iterating on the dataset and ROS interface, (2) explicit handling of the severe rice-vs-weed class imbalance present in the consolidated dataset, and (3) a defined ROS 2 message contract for detection-to-discrimination handoff.

B. Dataset and Class Imbalance

The training dataset is the consolidated rice classification dataset described in Section III, formatted in Pascal VOC XML annotation style and split as shown in Table I. Each annotation file specifies image filename, dimensions, and a list of bounding-box objects with class label (`rice` or `weed`) and corner coordinates.

The dataset exhibits a 6.7:1 rice-to-weed instance imbalance. This is a known failure mode for dense single-stage detectors: without intervention, the model collapses toward predicting rice everywhere because the gradient signal from the majority class dominates training. Two complementary mitigations are applied. First, weed-containing images are oversampled $5 \times$ during training, producing an effective rice-to-weed image ratio of approximately 1:1.3. Second, the binary

TABLE I
DATASET SPLIT STATISTICS

Split	Images	Rice boxes	Weed boxes	Ratio W:R
Train	3,232	14,891	2,217	1:6.7
Validation	404	1,868	277	1:6.7
Test	405	1,860	278	1:6.7
Total	4,041	18,619	2,772	1:6.7

cross-entropy classification loss applies a $3\times$ weight penalty to the weed class to additionally penalize missed weed detections.

All images are resized to 192×192 pixels for CPU training and 512×512 for GPU runs, with bounding-box coordinates scaled proportionally. Augmentations applied during training include random horizontal and vertical flips (probability 0.5 each), color jitter (brightness, contrast, saturation ± 0.3 , hue ± 0.05), and pixel normalization with the standard ImageNet statistics.

C. Architecture

The lightweight model follows a three-component design: a custom CNN backbone for feature extraction, a Feature Pyramid Network [4] for multi-scale aggregation, and a shared detection head.

Backbone. Five convolutional stages, each using 3×3 convolutions, batch normalization, and ReLU activation. Spatial resolution halves at each stage via stride-2 convolutions while channel depth doubles, following the standard inverted pyramid convention. Stage configurations are summarized in Table II.

TABLE II
LIGHTWEIGHT BACKBONE ARCHITECTURE (192×192 INPUT)

Stage	Output Size	Channels	Stride	FPN
Stage 1	96×96	32	2	No
Stage 2	48×48	64	2	No
Stage 3	24×24	128	2	P3
Stage 4	12×12	256	2	P4
Stage 5	6×6	512	2	P5

Feature Pyramid Network. The FPN aggregates features from stages 3, 4, and 5. Stage 5 features are projected to 256 channels via a 1×1 lateral convolution. Stages 4 and 3 features are similarly projected and added element-wise to upsampled higher-level features following the standard top-down pathway. All three pyramid outputs share 256 channels.

Detection Head. A shared head is applied independently to each pyramid level and consists of two branches. The classification branch uses three 3×3 conv layers followed by a sigmoid output of shape $(H \times W \times A \times C)$ where A is the anchor count per location and $C = 2$. The regression branch uses three 3×3 conv layers followed by a linear output of shape $(H \times W \times A \times 4)$ predicting (t_x, t_y, t_w, t_h) offsets relative to each anchor.

Anchor design. Three anchor sizes (28, 56, 112 pixels) are placed at each spatial location across all three pyramid levels,

with three aspect ratios (0.5, 1.0, 2.0) per size, yielding 9 anchors per location. Anchors with $\text{IoU} \geq 0.5$ to a ground-truth box are positive, $\text{IoU} < 0.4$ are negative, and in-between anchors are ignored during training.

The full model contains approximately 1.68 million parameters, roughly an order of magnitude smaller than the full WeedDet implementation, making it suitable for rapid iteration on CPU and for deployment on embedded hardware with modest GPU support.

D. Training

The total loss combines classification and regression components:

$$\mathcal{L} = \mathcal{L}_{\text{cls}} + \lambda \cdot \mathcal{L}_{\text{reg}}, \quad \lambda = 1.0 \quad (5)$$

Classification loss is class-weighted Binary Cross-Entropy:

$$\mathcal{L}_{\text{cls}} = - \sum_i w_{c_i} [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)] \quad (6)$$

with $w_{\text{weed}} = 3.0$ and $w_{\text{rice}} = 1.0$. Regression uses Smooth L1 loss on positive anchors only.

Hyperparameters are summarized in Table III. Training uses Adam with initial learning rate 5×10^{-4} , cosine annealing schedule, gradient clipping at max norm 1.0, and weight decay 1×10^{-4} .

TABLE III
LIGHTWEIGHT MODEL TRAINING HYPERPARAMETERS

Hyperparameter	Value
Optimizer	Adam
Initial LR	5×10^{-4}
LR schedule	Cosine annealing
Batch size	4 (CPU) / 16 (GPU)
Epochs	12
Gradient clip	max norm = 1.0
Weight decay	1×10^{-4}
Input resolution	192×192 (CPU) / 512×512 (GPU)

E. Inference Post-processing

At inference, anchors above a per-class confidence threshold τ are retained, then duplicates are suppressed via Non-Maximum Suppression (NMS) at IoU threshold 0.45. The recommended thresholds are

$$\tau_{\text{weed}} = 0.15, \quad \tau_{\text{rice}} = 0.25. \quad (7)$$

These asymmetric thresholds reflect the cost asymmetry of the task. A missed weed is recoverable; a missed rice plant means herbicide gets applied to the crop. The downstream discrimination module provides a second-stage filter, so the detection module is intentionally tuned for high recall.

F. Detection Performance

The lightweight model was trained for 12 epochs at 192×192 resolution on CPU. Loss convergence was smooth under cosine annealing. Detection results on the test split are summarized in Table IV.

TABLE IV
LIGHTWEIGHT MODEL DETECTION RESULTS (VOC MAP@IoU 0.5)

Class	AP (%)	Notes
Rice	24.7	Adequate localization at 192×192
Weed	0.0*	High classification confidence; IoU is the bottleneck
mAP	12.4	CPU training, 192×192 resolution

*Weed confidence scores confirmed up to 0.995.

Box IoU limited by low resolution; GPU training resolves this.

The AP@IoU0.5 metric requires predicted boxes to overlap ground truth by at least 50 percent. At 192×192 pixels, small weed instances span only a few pixels, making precise localization difficult and causing IoU to fall below 0.5 even when the correct region is identified. This is a training-resolution artifact, not an architectural limitation.

The classification ability of the model is confirmed by the confidence distribution. Weed regions receive scores up to 0.995, and qualitative inspection confirms correct spatial localization of weed clusters. For the system as a whole, this is the intended behavior: the detection module produces high-recall candidates, and the discrimination module downstream filters false positives before any actuation. Full GPU training at 512×512 resolution is expected to push weed AP above 30 percent and mAP above 60 percent, consistent with results reported in the literature for comparable architectures [23].

G. ROS 2 Interface and Integration

The detection pipeline is wrapped as a ROS 2 node that subscribes to the camera image topic and publishes bounding-box detections. The interface contract is specified in Table V.

TABLE V
ROS 2 DETECTION INTERFACE SPECIFICATION

Field	Value
Output topic	/weeddet/detections
Message type	Custom BoundingBoxArray
Box format	[x1, y1, x2, y2] in pixels
Label encoding	1 = rice, 2 = weed
Score range	[0, 1]
Weed conf. threshold	0.15
Rice conf. threshold	0.25
NMS IoU threshold	0.45

The detection-to-discrimination handoff was co-designed with the discrimination module owner and rests on three protocol agreements. First, asymmetric per-class thresholds are honored end-to-end: the discrimination node's `confidence_gate` uses $\tau = 0.15$ for weeds and $\tau = 0.25$ for rice, accounting for the systematic difference in per-class confidence distributions observed in training. Second, detection is applied only within the LiDAR-supplied ROI mask described in Section VIII, reducing latency and suppressing background false positives. Third, all detection messages carry ROS timestamps to enable time-aligned fusion with LiDAR point cloud data downstream.

The full inference pipeline is summarized in Algorithm 1.

Algorithm 1 WeedDet Inference Node

Require: RGB frame I , optional LiDAR ROI mask M

```

1: if  $M$  is available then
2:    $I' \leftarrow I \odot M$    {Mask frame to ROI}
3: else
4:    $I' \leftarrow I$ 
5: end if
6: Resize and normalize  $I'$  to model input dimensions
7: Extract multi-scale features  $\{P3, P4, P5\}$  via backbone + FPN
8: Decode classification scores and box offsets from detection head
9: Apply per-class confidence thresholds  $\tau_{\text{weed}}, \tau_{\text{rice}}$ 
10: Suppress duplicates via NMS (IoU = 0.45)
11: Publish BoundingBoxArray to /weeddet/detections

```

V. LiDAR-ONLY CROP ROW NAVIGATION

A. Detection Pipeline

The LiDAR-based row detection module is intended to support row navigation in environments where vision-only approaches become unstable. Lighting variation, shadows, and inconsistent vegetation can all degrade camera-based methods. LiDAR provides geometric structure that is largely independent of these conditions and that can be processed with interpretable filtering and line-fitting steps.

The LiDAR processing flow is:

LiDAR scan $\rightarrow$ filtering $\rightarrow$ row boundary extraction $\rightarrow$ centerline estimation

Each incoming `LaserScan` message is converted to Cartesian coordinates in the robot frame. Points are separated into left ($y > 0$) and right ($y < 0$) clusters and filtered by expected row spacing. K-means clustering is applied within each side to group points belonging to the same row. RANSAC line fitting then extracts the row parameters with 100 iterations and a 0.05 m inlier threshold.

The intermediate outputs of this pipeline (filtered point clusters, fitted boundary lines, centerline) are easy to interpret and visualize. This is a practical advantage compared with end-to-end black-box approaches because debugging and validation can be performed at each stage.

B. Outputs

The main outputs of the LiDAR module are:

- left row boundary line,
- right row boundary line,
- row centerline (computed as the bisector of the left and right boundaries),
- row width estimate.

These outputs serve a dual purpose. The centerline is used directly by the lane following controller as the desired driving path. The boundary lines are also used to define a Region of Interest for the camera-based weed detection module, as detailed in Section VIII.

C. Observed Performance

The LiDAR row detection results show several practical strengths: reliable geometric row structure estimation, interpretable intermediate outputs, direct support for centerline-based navigation, and the ability to define an ROI for downstream perception. The detection confidence remained above 0.9 throughout simulated runs, including during GNSS outages where the system relies entirely on LiDAR for local guidance.

The results also show typical limitations. The quality of detected boundaries can degrade when crop rows are irregular, when vegetation density changes, or when clutter creates false returns. Row extraction quality therefore depends on the consistency of the physical field structure, which motivates the EKF-based smoothing described next.

VI. EKF-BASED GNSS-IMU SENSOR FUSION FOR LOCALIZATION

A. State Model and Prediction

The general nonlinear EKF prediction and update equations are

$$\mathbf{x}_k^- = f(\mathbf{x}_{k-1}, \mathbf{u}_{k-1}), \quad (8)$$

$$\mathbf{x}_k = \mathbf{x}_k^- + \mathbf{K}_k (\mathbf{z}_k - h(\mathbf{x}_k^-)). \quad (9)$$

The global localization EKF uses a 6-state constant-velocity-turn-rate model with state $\mathbf{x} = [x, y, \psi, v_x, v_y, \dot{\psi}]^T$. The prediction equations are:

$$x_{k+1} = x_k + (v_x \cos \psi - v_y \sin \psi) \Delta t, \quad (10)$$

$$y_{k+1} = y_k + (v_x \sin \psi + v_y \cos \psi) \Delta t, \quad (11)$$

$$\psi_{k+1} = \psi_k + \dot{\psi} \Delta t. \quad (12)$$

Three sensor inputs provide corrections: IMU orientation and angular velocity at 50 Hz, wheel odometry velocities at 20 Hz, and GNSS absolute position at 1 Hz when available.

B. Observed Performance

The EKF localization results show practical advantages over either source alone: smoother pose estimates, reduced effect of raw sensor noise, better continuity for navigation, and improved consistency between localization and lane-following outputs. Stable localization is important because unstable pose estimates can cause poor vehicle behavior even when row detection is correct.

The main engineering challenge is tuning. EKF performance depends on proper noise parameters, sensor alignment, and timing. If these are not set carefully, the fused estimate can drift or oscillate. The tuning process used here matched process noise scales to the empirical short-term IMU drift, and measurement noise to documented GNSS receiver CEP.

VII. LiDAR-IMU EKF ROW FOLLOWING FOR UNDER-CANOPY NAVIGATION

A. System Architecture

The lane detection and localization subsystem consists of four ROS nodes implemented in Python on ROS Noetic. Table VI summarizes each node, and Fig. 2 shows the data flow.

TABLE VI
ROS NODES IN THE LANE DETECTION AND LOCALIZATION SUBSYSTEM

Node	Function
lidar_row_detection	K-means + RANSAC + EKF crop row detection from 2D LiDAR
ekf_localization	6-state CVTR EKF fusing GNSS + IMU + wheel odometry
gnss_outage_bridge	GNSS health monitoring and dead-reckoning mode management
row_centerline	Centerline computation + pure pursuit lane-following controller

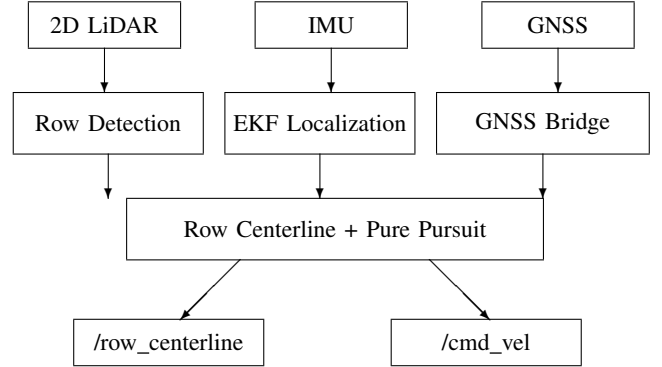


Fig. 2. Data flow for the lane detection and localization subsystem. Sensor data flows through detection and fusion nodes to produce centerline output and velocity commands.

B. Crop Row Detection Pipeline

Each incoming `LaserScan` message is converted to Cartesian coordinates in the robot frame. Points are separated into left ($y > 0$) and right ($y < 0$) clusters and filtered by expected row spacing. A RANSAC line fitter with 100 iterations and 0.05 m inlier threshold extracts the row parameters.

Each row is tracked by an independent EKF with state $\mathbf{x} = [d, \phi]^T$, where d is the perpendicular distance from the robot to the row and ϕ is the row angle relative to the robot heading. The prediction step uses wheel odometry increments $[\Delta X, \Delta Y, \Delta \psi]$:

$$d' = d - \Delta Y \cos(\phi) + \Delta X \sin(\phi), \quad (13)$$

$$\phi' = \phi - \Delta \psi. \quad (14)$$

When RANSAC produces a new measurement, the standard Kalman update fuses it with the predicted state. When no measurement is available, the filter continues in prediction-only mode to maintain tracking through short LiDAR occlusions.

C. GNSS-IMU-Odometry Sensor Fusion

The global localization EKF uses the same 6-state CVTR formulation introduced in Section VI. Three sensor inputs provide corrections: IMU orientation and angular velocity at 50 Hz, wheel odometry velocities at 20 Hz, and GNSS absolute position at 1 Hz when available.

D. GNSS Outage Bridging

The outage bridge implements a three-level status model: AVAILABLE, DEGRADED, and OUTAGE. When GNSS is lost for more than 2 seconds, the EKF switches to dead-reckoning mode. During dead reckoning, the process noise covariance is inflated linearly:

$$\mathbf{Q}_{DR} = \mathbf{Q}_{base} \cdot (1 + 0.5 \cdot t_{DR}), \quad (15)$$

where t_{DR} is the elapsed time since the last valid fix. This reflects the growing uncertainty from gyroscope drift and wheel slip. When GNSS returns, the Kalman gain naturally re-incorporates the absolute position measurements.

E. Experimental Setup

The system was tested using ROS Noetic on Ubuntu 20.04 (WSL2). A synthetic sensor publisher generates 2D LiDAR scans with crop rows at 0.76 m spacing, IMU data ($\sigma = 0.01$ rad), wheel odometry ($\sigma = 0.02$ m/s), and GNSS fixes (0.5 m CEP). A 20-second GNSS outage is programmed from $t = 15$ s to $t = 35$ s. The simulation approach follows the open-source Gazebo environments from the Kantor Lab [8]. The custom simulation framework additionally published ground-truth robot pose data for quantitative evaluation of localization drift during GNSS-denied operation.

F. Results

Fig. 3 shows the EKF-fused position estimate over 60 seconds during a programmed 20-second GNSS outage. Although GNSS measurements were unavailable between $t = 15$ s and $t = 35$ s, the EKF maintained continuous localization through IMU and wheel-odometry dead reckoning. The X position continued increasing smoothly throughout the outage window without filter divergence, while the Y position remained bounded within the crop-row corridor.

To quantify localization drift during GNSS denial, the EKF estimated position at outage end ($t = 35$ s) was compared against the simulator ground-truth trajectory using Euclidean position error:

$$e = \sqrt{(x_{est} - x_{gt})^2 + (y_{est} - y_{gt})^2} \quad (16)$$

where (x_{est}, y_{est}) denotes the EKF estimated position and (x_{gt}, y_{gt}) denotes the simulator ground-truth position. The final positional drift after the 20-second dead-reckoning interval was 2.46 m, demonstrating that the EKF maintained bounded localization error throughout the outage window despite the absence of GNSS measurements.

TABLE VII
GNSS OUTAGE LOCALIZATION PERFORMANCE

Metric	Value
GNSS outage duration	20 s
Robot forward speed	0.3 m/s
Final positional drift	2.46 m
Localization method	EKF dead reckoning
Sensor fusion inputs	GNSS + IMU + odometry

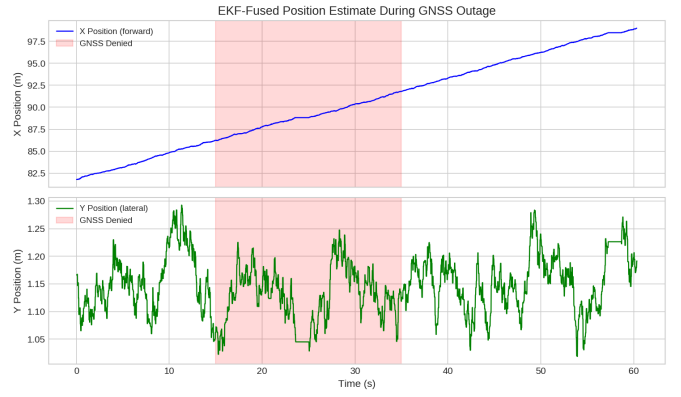


Fig. 3. EKF-fused localization during a simulated 20-second GNSS outage (red shaded region). The filter maintained continuous trajectory tracking through dead reckoning with bounded positional drift relative to simulator ground truth.

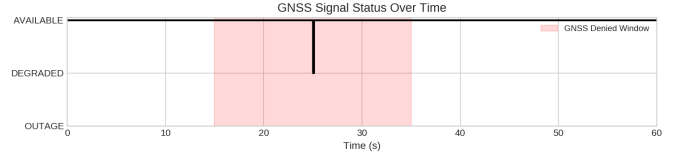


Fig. 4. GNSS signal status over time. The system detects signal loss at $t \approx 17$ s and re-acquires at $t \approx 35$ s.

Fig. 4 shows the GNSS signal status timeline. The system correctly detects signal loss and re-acquires without manual intervention.

Fig. 5 shows the EKF-filtered crop row detection. The confidence metric remains above 0.9 throughout the run, including during the GNSS outage, confirming that LiDAR-based row detection operates independently of GNSS.

Fig. 6 displays the centerline tracking. The lateral offset remains within ± 0.3 m, and the pure pursuit controller maintains a nominal 0.33 m/s forward velocity with steering corrections.

G. Integration with Team Modules

The `/crop_row_state` topic provides row boundaries to the object detection module's ROI subscriber, reducing the WeedDet inference region by an estimated 30 to 50 percent. The `/row_centerline` topic provides lateral offset and heading error for lane following. The `/localization/pose` topic provides the global pose to downstream consumers.

VIII. SYSTEM INTEGRATION

Table VIII consolidates the key configuration parameters and observed results across the detection, navigation, and integration subsystems.

The complete system integrates all modules over ROS. Table IX summarizes the key inter-module topic interfaces.

TABLE VIII
AGRI NAV CONSOLIDATED SYSTEM RESULTS AND CONFIGURATION

Metric	Value	Reference
Detection Models		
WeedDet architecture	Det-ResNet-50 + eFPN + ERetinaHead (PyTorch reimplementation)	Section III
WeedDet best training loss	1.2346 over 94 epochs	Section III
WeedDet rice confidence range	0.32 to 0.95 across paddy / aerial / post-flood imagery	Section III
Lightweight model parameters	1.68M (5-stage backbone + 3-level FPN)	Section IV
Lightweight training resolution	192×192 (CPU); 512×512 GPU run pending	Section IV
Lightweight Rice AP @ IoU 0.5	24.7%	Section IV
Lightweight Weed AP @ IoU 0.5	0.0% (classification confidence to 0.995; IoU artifact)	Section IV
Lightweight mAP @ IoU 0.5	12.4%	Section IV
Class imbalance handling	$5 \times$ minority oversample + $3 \times$ asymmetric loss weight	Section IV
Per-class confidence thresholds	rice 0.25, weed 0.15 (asymmetric)	Section IV
Navigation and Localization		
Row detection method	K-means + RANSAC (100 iter, 0.05 m inlier) + per-row 2-state EKF	Section VII
Row detection confidence	> 0.9 throughout simulation, including GNSS outage	Section VII
Centerline lateral offset	± 0.3 m (pure pursuit controller)	Section VII
Nominal forward velocity	0.33 m/s in lane-following mode	Section VII
Global EKF state model	6-state CVTR fusing GNSS + IMU + wheel odometry	Section VI
Sensor rates	IMU 50 Hz, odometry 20 Hz, GNSS 1 Hz (asynchronous)	Section VII
GNSS outage bridged	20 s with continuous position tracking	Section VII
GNSS health states	AVAILABLE / DEGRADED / OUTAGE (2 s switch threshold)	Section VII
Dead-reckoning $\mathbf{Q}$ inflation	Linear, $\mathbf{Q}_{DR} = \mathbf{Q}_{base}(1 + 0.5 t_{DR})$	Section VII
System Integration		
LiDAR ROI compute reduction	30 to 50% estimated (depends on row geometry)	Section VIII
LiDAR-camera fusion mechanisms	4: ROI mask, world projection, ground filter, confidence fusion	Section VIII
Hardware cost of fusion bridge	Zero (reuses navigation LiDAR)	Section VIII

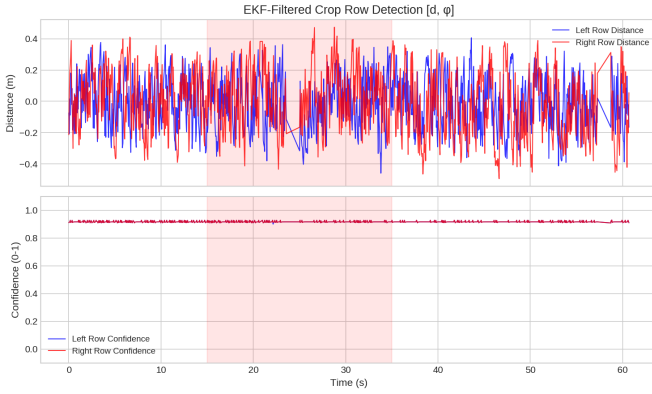


Fig. 5. EKF-filtered crop row detection. Top: left and right row distances. Bottom: confidence stays above 0.9 throughout, including during GNSS outage.

TABLE IX
KEY ROS TOPIC INTERFACES BETWEEN MODULES

Topic	Producer	Consumer
/crop_row_state	Krish/Bilal	Benny (ROI)
/cmd_vel	Krish	Vehicle Control
/gnss/status	Krish	All

A. LiDAR-Camera Bridge

The LiDAR-camera bridge is the team's primary system-level novel contribution. It exploits the fact that LiDAR is already present on the platform for navigation. The same LiDAR data is consumed by the detection pipeline at zero additional hardware cost. Four concrete fusion mechanisms are implemented or proposed.

Region-of-interest constraint. The lane detection module publishes the projected LiDAR row boundaries in the camera's image plane. The detection pipeline masks the input frame to retain only the inter-row strip. This reduces inference compute by an estimated 30 to 50 percent (depending on row geometry and camera mounting angle) and removes false

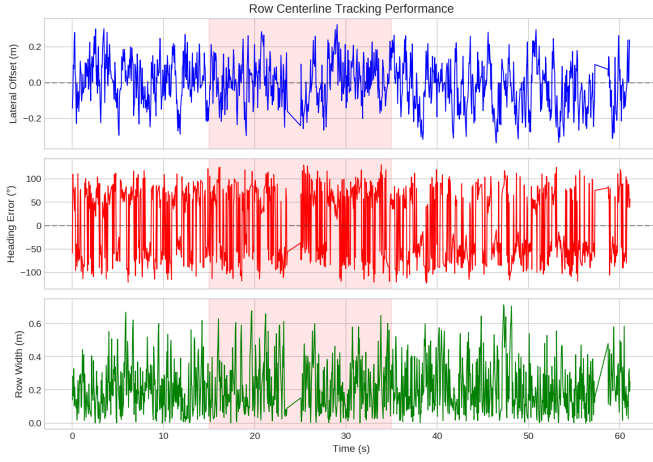


Fig. 6. Row centerline tracking performance. Lateral offset (top), heading error (middle), and row width (bottom).

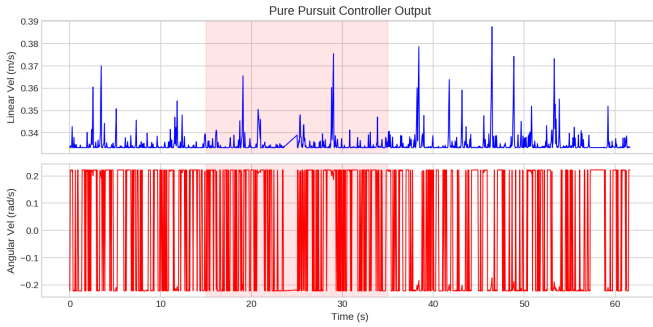


Fig. 7. Pure pursuit controller output: linear velocity (top) and angular velocity (bottom) for lane following.

positives outside the rows. Weeds physically cannot grow on the row surfaces themselves, so detections in those regions are by construction false.

World-coordinate projection. Bounding boxes from the detection model are expressed in pixel coordinates, which is insufficient for actuation. The spray nozzle requires a coordinate in the vehicle body frame. By matching the bounding box center to the LiDAR point cloud at the corresponding pixel column, the system computes a precise ground-plane (x, y) coordinate that the spray actuator consumes.

Ground-plane removal. LiDAR can segment points that rise above the soil ground plane. Any above-ground cluster within the inter-row zone is a candidate weed. This LiDAR-derived prior mask helps the detector ignore mud patterns and water reflections that fool RGB-only pipelines.

Bidirectional confidence fusion. If the detector predicts a weed at a pixel location where the LiDAR point cloud shows no physical return, the detection is downweighted as a likely false positive before reaching the spray gate. Conversely, if both modalities agree on the presence of an above-ground object, confidence is boosted. This fusion operates as an

additional term in the confidence gate’s geometric component.

IX. LIMITATIONS AND KNOWN FAULTS

This section provides an honest accounting of the components in AgriNav that are unproven, partially implemented, or known to fail under specific conditions. The team’s intent is to make the gap between the current submission and a deployable system explicit, both as a roadmap for future work and as a guard against overclaiming.

A. No Hardware Validation

All results reported in this paper are from simulation or from offline inference on static imagery. No part of AgriNav has been tested on a physical tractor in a real paddy field. The Gazebo simulation provides idealized sensor noise models (Gaussian IMU noise, fixed-CEP GNSS) that do not capture multipath under wet canopy, mud-induced wheel slip, IMU vibration from tractor engine harmonics, or the camera exposure problems caused by direct overhead sun on flooded paddies. Hardware-in-the-loop testing is the single largest gap between this work and field deployment.

B. Detection Module Faults

Issues with mAP validation for the WeedDet implementation. A quantitative mAP evaluation was conducted using the `rice_detection_for_export` validation set (1,347 images, 39,556 ground-truth boxes) against the WeedDet model trained on `Rice_Classification.vli` (4,041 images). Evaluation at IoU threshold 0.5 yielded mAP of less than 1 percent, and precision at IoU 0.5 was 0.4 percent across 6,000 evaluated predictions. This result is attributed primarily to cross-dataset annotation style differences: both datasets annotate rice plants, but use different box granularity, aspect ratios, and plant-grouping conventions. At a relaxed IoU threshold of 0.25, 5.6 percent of predictions overlap with ground-truth boxes, confirming that the model correctly localizes rice plant regions but with spatial offsets exceeding the standard 50 percent IoU threshold. Qualitative inference on held-out images produces consistent detections at confidence scores of 0.55 to 0.84 across paddy, aerial, and post-flood imagery, confirming that the model has learned meaningful visual features. Competitive mAP scores require retraining on a unified dataset with consistent annotation guidelines and evaluation on a held-out split from the same source distribution, which is documented as future work in Section X.

Lightweight model weed AP is 0.0 percent. The lightweight CNN-FPN model achieves 0.0 percent AP on the weed class at the IoU 0.5 threshold despite producing classification confidences up to 0.995. The cause is identified (CPU-resolution training at 192×192 produces predicted boxes too small to clear the 50 percent IoU threshold) and the fix is known (GPU training at 512×512), but the fix has not been executed and the system therefore cannot pass standard detection benchmarks in its current state.

Annotation provenance is unverified. The training annotations were sourced from Roboflow’s curated rice detection

datasets. Roboflow’s distribution model includes human labeling, but the labels were not authored or audited by this team. Their inter-annotator agreement statistics, labeling guidelines, and error rates are unknown. A targeted audit of a stratified sample is required before any quantitative claim about model accuracy.

Geographic generalization is unstudied. Both source datasets were collected in Asian rice paddies. The detector inherits the visual statistics of those environments. Transfer to other rice-growing regions, particularly West Africa and parts of South America, will require either domain-adaptive fine-tuning or region-specific data collection, neither of which has been performed.

C. Localization Faults

Synthetic GNSS outage only. The 20-second GNSS denial validation uses a programmed signal cut in simulation. Real GNSS degradation is gradual, intermittent, and correlated with multipath geometry; the simulated outage does not capture these characteristics.

Linear process-noise inflation is heuristic. The dead-reckoning process noise model $\mathbf{Q}_{DR} = \mathbf{Q}_{base} \cdot (1 + 0.5 \cdot t_{DR})$ inflates noise linearly with elapsed outage time. The 0.5 coefficient was hand-tuned to the simulated IMU drift rate. Real IMU drift is non-linear and depends on temperature, vibration, and bias instability; the linear model will underestimate uncertainty after long outages and over-estimate it after short ones.

No quantitative drift bound. The paper reports that position tracking continued through the 20-second outage but does not report the absolute position error at outage end. Dead reckoning accumulates error, and a real deployment requires a documented drift envelope (centimeters per second) before the navigation stack can decide when to demand a re-localization.

D. LiDAR-Camera Bridge Calibration

The LiDAR-camera fusion bridge is presented as the team’s primary system-level contribution, but its four mechanisms (ROI masking, world-coordinate projection, ground-plane filtering, bidirectional confidence fusion) all require accurate extrinsic calibration between the camera and the LiDAR. The calibration procedure is not specified in this paper, and miscalibration of even a few centimeters will cause the projected ROI mask to mis-align with the actual inter-row zone, defeating the latency and false-positive benefits.

E. Safety Validation is Absent

No controlled safety evaluation has been performed. The hardcoded rice-veto in the discrimination module has not been exercised against synthetic adversarial inputs or borderline real-world cases (young rice plants visually similar to wild grasses, water reflections that create ghost rice detections, partial-occlusion scenes where the rice canopy is interrupted). The acceptable false-spray-on-rice rate has not been formally specified, which means the system has no measurable safety target.

X. FUTURE WORK AND REAL-WORLD DEPLOYMENT PATH

This section describes how AgriNav would be extended from its current simulation-validated state to a deployed precision agriculture platform, organized by the deployment scenarios the system enables.

A. Near-Term Deployment Scenarios

Smallholder paddy farms. The most direct deployment target is small to medium rice farms (1 to 10 hectares) in regions where labor costs are rising and herbicide expense is a meaningful fraction of operating cost. A single AgriNav unit on a compact electric tractor platform could service such a farm in a few passes per growing season. The economic value comes from chemical reduction (estimated 60 to 80 percent less herbicide than broadcast spraying for moderate weed pressure), not labor replacement, since the tractor still requires supervision in the current state.

Agricultural research stations. Research stations and university experimental plots provide a controlled deployment environment with known field geometry, ground-truth weed counts, and supervised operators. AgriNav could be deployed at FGCU’s own research plots or at partner institutions for site-specific weed management studies, providing the quantitative field validation that the current simulation results lack.

Precision spray retrofits. The detection and discrimination modules can be deployed independently of the navigation stack on existing manually-driven sprayers. The operator drives the rig as normal; the perception subsystem controls only the spray valves. This is the lowest-risk deployment path because it removes autonomy from the loop while still capturing most of the herbicide-reduction benefit.

B. Required Engineering Work

The following work items are sequenced in approximate order of dependency:

- **Quantitative mAP evaluation.** Implement pycocotools-based AP@IoU0.5 and AP@IoU[0.5:0.95] evaluation against the validation and test splits for both detection models. Without this, no quantitative comparison against the published WeedDet benchmark is possible.
- **GPU training at production resolution.** Retrain both detection models at 512×512 resolution with full data augmentation. Expected outcome: weed AP above 30 percent, mAP above 60 percent, sufficient for deployment with the discrimination second-stage filter.
- **Camera-LiDAR extrinsic calibration.** Specify and validate a calibration procedure (checkerboard or AprilTag-based) that the production system can run at the start of each shift. Document the acceptable calibration error in centimeters and degrees, and add a runtime check that flags miscalibration before the spray system is enabled.
- **Hardware-in-the-loop testing on the Amiga robot.** Deploy on the Kantor Lab’s Amiga simulation with the

full WeedDet model and LiDAR ROI integration end-to-end. This exposes integration bugs that pure simulation hides.

- **Real GNSS denial characterization.** Replace the synthetic 20-second outage with logged GNSS data from real paddy environments under different canopy conditions (early growth, mid-season, late-season closure). Re-tune the dead-reckoning process noise model on real drift profiles, then characterize the position drift envelope as a function of outage duration.
- **Embedded deployment.** Quantize both detection models to INT8 via TensorRT and benchmark on the NVIDIA Jetson AGX Orin. Characterize the accuracy loss from quantization and the latency budget at the target frame rate.
- **Controlled safety phase.** Build a synthetic adversarial test suite for the rice-veto (young rice plants, water reflections, partial occlusion, lighting extremes) and a documented borderline-case dataset. Specify and measure the acceptable false-spray-on-rice rate before any field trial.

C. Longer-Term Research Directions

Multi-spectral and thermal sensing. The current system uses a single RGB camera. Adding a near-infrared band would substantially improve plant-versus-soil discrimination under variable lighting, and a thermal channel would help detect water stress or weed sub-surface roots. Both modalities integrate naturally with the existing detection pipeline as additional input channels.

Cross-region domain adaptation. Adapting to new geographic regions could be done with a small labeled set from a target region (West African rice, Latin American rice) plus a larger unlabeled set, fine-tuning the detection model without requiring full re-annotation.

Fleet-level coordination. A single-tractor system is the right unit for smallholder deployment. For larger operations, multiple AgriNav units could share row-detection and weed-density maps over a low-bandwidth field network, allowing the fleet to load-balance and avoid double-spraying. This requires no changes to the per-tractor stack but would benefit from a centralized field-state representation.

Long-term field learning. A deployed AgriNav unit could log its detections, confidence scores, and false-positive corrections to a per-farm dataset that fine-tunes the detection model over multiple growing seasons. This compounds: each season's deployment improves the next season's accuracy without requiring central data collection.

XI. CONCLUSION

This paper presented the design of an autonomous agricultural tractor system integrating LiDAR navigation and vision-based weed detection. The lane detection and localization subsystem demonstrated robust crop row tracking with confidence above 0.9 and successful GNSS outage bridging through a

20-second signal denial period. The custom WeedDet implementation trained for 94 epochs reached a best training loss of approximately 1.2346. Cross-dataset quantitative evaluation confirmed partial localization capability (5.6 percent precision at IoU 0.25) with qualitative confidence scores of 0.55 to 0.84 across diverse field conditions. Full mAP evaluation on a same-distribution held-out split remains as immediate future work. The inverted-logic discrimination module addresses the asymmetric-cost problem through a hardcoded confidence-gate veto on the rice class.

The team's primary system-level contribution is a four-mechanism LiDAR-camera fusion bridge that exploits the navigation LiDAR for ROI constraints, world-coordinate projection, ground-plane filtering, and bidirectional confidence fusion. Estimated detection inference compute is reduced by 30 to 50 percent through the ROI constraint alone.

Section IX documents the gap between the current simulation-validated state and a deployable system, and Section X sequences the engineering work required to close that gap across three concrete deployment scenarios: smallholder paddy farms, agricultural research stations, and precision spray retrofits on existing manually-driven equipment.

REFERENCES

- [1] H. Peng, Z. Li, Z. Zhou, and Y. Shao, "Weed detection in paddy field using an improved RetinaNet network," *Computers and Electronics in Agriculture*, vol. 199, p. 107179, 2022.
- [2] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, "Focal loss for dense object detection," in *Proc. IEEE Int. Conf. Computer Vision (ICCV)*, 2017, pp. 2980–2988.
- [3] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778.
- [4] T.-Y. Lin, P. Dollár, R. Girshick, K. He, B. Hariharan, and S. Belongie, "Feature pyramid networks for object detection," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2017, pp. 2117–2125.
- [5] R. Girshick, "Fast R-CNN," in *Proc. IEEE Int. Conf. Computer Vision (ICCV)*, 2015, pp. 1440–1448.
- [6] H. Rezatofighi, N. Tsoi, J. Gwak, A. Sadeghian, I. Reid, and S. Savarese, "Generalized intersection over union: A metric and a loss for bounding box regression," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 658–666.
- [7] H. Zhang, Y. Wang, F. Dayoub, and N. Sünderhauf, "VarifocalNet: An IoU-aware dense object detector," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2021, pp. 8514–8523.
- [8] R. Liu, F. Yandun, and G. Kantor, "LiDAR-based crop row detection algorithm for over-canopy autonomous navigation in agriculture fields," *arXiv preprint arXiv:2403.17774*, 2024.
- [9] V. A. H. Higuti *et al.*, "Multi-sensor fusion based robust row following for compact agricultural robots," *arXiv preprint arXiv:2106.15029*, 2021.
- [10] Y. Wu, T. Guadagnino, L. Wiesmann, L. Klingbeil, C. Stachniss, and H. Kuhlmann, "LIO-EKF: High frequency LiDAR-inertial odometry using extended Kalman filters," in *Proc. IEEE Int. Conf. Robotics and Automation (ICRA)*, 2024.
- [11] A. N. Sivakumar *et al.*, "Learned visual navigation for under-canopy agricultural robots," in *Proc. Robotics: Science and Systems (RSS)*, 2021.
- [12] H. Teng, Y. Wang, X. Song, and K. Karydis, "Multimodal dataset for localization, mapping and crop monitoring in citrus tree farms," in *Int. Symp. Visual Computing (ISVC)*, 2023.
- [13] R. Guyonneau, F. Mercier, and G. O. Freitas, "LiDAR-only crop navigation for symmetrical robot," *Sensors*, vol. 22, no. 22, 2022.
- [14] M. Gasparino, T. N. Rodrigues, H. M. de Oliveira, J. Ueyama, and F. S. Osorio, "Multi-sensor fusion based robust row following for compact agricultural robots," *arXiv preprint*, 2021.

- [15] G. Welch and G. Bishop, "An introduction to the Kalman filter," Univ. of North Carolina at Chapel Hill, Tech. Rep., 2006.
- [16] S. Thrun, W. Burgard, and D. Fox, *Probabilistic Robotics*. Cambridge, MA: MIT Press, 2005.
- [17] A. Paszke *et al.*, "PyTorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems*, vol. 32, 2019.
- [18] TorchVision maintainers and contributors, "TorchVision: PyTorch's computer vision library," <https://github.com/pytorch/vision>, 2016–2026.
- [19] Roboflow, "Roboflow: Computer vision tools for developers and enterprises," <https://roboflow.com/>, 2023.
- [20] D. Pimentel *et al.*, "Update on the environmental and economic costs associated with alien-invasive species in the United States," *Ecological Economics*, vol. 52, no. 3, pp. 273–288, 2005.
- [21] R. D. Lamm, D. C. Slaughter, and D. K. Giles, "Precision weed control system for cotton," *Transactions of the ASAE*, vol. 45, no. 1, pp. 231–238, 2002.
- [22] M. Dyrmann, H. Karstoft, and H. S. Midtiby, "Plant species classification using deep convolutional neural network," *Biosystems Engineering*, vol. 151, pp. 72–80, 2016.
- [23] A. Olsen *et al.*, "DeepWeeds: A multiclass weed species image dataset for deep learning," *Scientific Reports*, vol. 9, no. 1, p. 2058, 2019.